

**МИНОБРНАУКИ РОССИИ**  
**САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ**  
**ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ**  
**«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)**  
**Кафедра МО ЭВМ**

**ИНДИВИДУАЛЬНОЕ ДОМАШНЕЕ ЗАДАНИЕ**  
**По дисциплине «Введение в нереляционные базы данных»**  
**Тема: Каталог хлеба с рецептами**

Студенты гр. 2383

Жданов А.К.

Лукьянова М.Р.

Мордасов Е.Д.

Руководитель

Заславский М.М.

Санкт-Петербург

2025

## ЗАДАНИЕ

Студенты

Жданов А.К.

Лукьянова М.Р.

Мордасов Е.Д.

Тема проекта: Разработка веб-сайта: «Каталог хлеба с рецептами»

Исходные данные:

Необходимо создать каталог видов хлеба, рецептов, отзывов, комментариев, обсуждений и рекомендаций. Используемая СУБД: MongoDB

Содержание пояснительной записки:

«Содержание»

«Введение»

«Сценарий использования»

«Модель данных»

«Разработанное приложение»

«Выводы»

«Приложение»

«Литература»

Предполагаемый объем пояснительной записки:

Не менее 10 страниц

Дата выдачи задания: 05.02.2025

Дата сдачи реферата: 20.05.2025

Дата защиты реферата: 20.05.2025

Студент гр. 2383

Жданов А.К.

Студент гр. 2383

Лукьянова М.Р.

Студентка гр. 2383

Мордасов Е.Д

## **АННОТАЦИЯ**

В рамках дисциплины «Введение в нереляционные базы данных» разработано приложение в команде. Выбрана тема: «Каталог хлеба с рецептами», используя базу данных MongoDB. Во внимание принимаются такие аспекты как производительность приложения и удобство пользования.

## **SUMMARY**

As part of the course "Introduction to Non-relational Databases", an application has been developed in a team. The chosen topic is "Bread Catalog with Recipes", using the MongoDB database. Such aspects as application performance and usability are considered.

## ОГЛАВЛЕНИЕ

|                                                                |    |
|----------------------------------------------------------------|----|
| ВВЕДЕНИЕ .....                                                 | 6  |
| 1. Актуальность решаемой проблемы .....                        | 6  |
| 2. Постановка задачи .....                                     | 6  |
| 3. Предлагаемое решение .....                                  | 7  |
| 4. Качественные требования к решению .....                     | 7  |
| СЦЕНАРИЙ ИСПОЛЬЗОВАНИЯ .....                                   | 8  |
| 1. Макет UI .....                                              | 8  |
| 2. Сценарий использования .....                                | 22 |
| 3. Преобладающие операции .....                                | 25 |
| Модель данных .....                                            | 26 |
| 1. Нереляционная модель данных .....                           | 26 |
| 2. Реляционная модель данных .....                             | 30 |
| 3. Сравнение моделей .....                                     | 34 |
| 4. Вывод .....                                                 | 35 |
| 3. РАЗРАБОТАННОЕ ПРИЛОЖЕНИЕ .....                              | 36 |
| 3.1. Краткое описание .....                                    | 36 |
| 3.2. Используемые технологии .....                             | 36 |
| 3.3. Снимки экрана приложения .....                            | 37 |
| ВЫВОДЫ .....                                                   | 38 |
| 4.1. Достигнутые результаты .....                              | 38 |
| 4.2. Недостатки и пути для улучшения полученного решения ..... | 38 |
| 4.3. Будущее развитие решения .....                            | 38 |
| ЗАКЛЮЧЕНИЕ .....                                               | 39 |
| ПРИЛОЖЕНИЯ .....                                               | 41 |
| ЛИТЕРАТУРА .....                                               | 42 |

## **ВВЕДЕНИЕ**

### **1. Актуальность решаемой проблемы**

Интерес к домашней выпечке и здоровому питанию продолжает расти, и всё больше людей ищут проверенные рецепты хлеба, делятся опытом и обсуждают особенности его приготовления. В этой связи становится актуальным создание веб-сайта, где пользователи смогут найти каталог различных видов хлеба, рецепты, оставить отзывы и участвовать в обсуждениях, формируя сообщество по интересам.

Для хранения данных проекта выбрана СУБД MongoDB, так как она хорошо подходит для работы с неструктурированной и гибкой по структуре информацией — рецептами, комментариями, изображениями и пользовательскими отзывами. MongoDB обеспечивает удобство масштабирования и высокую производительность, что делает её оптимальным выбором для разработки современного веб-приложения.

### **2. Постановка задачи**

Необходимо разработать веб-сайт с каталогом видов хлеба, содержащим рецепты, отзывы, комментарии и рекомендации. Пользователи должны иметь возможность удобно просматривать рецепты, делиться мнениями и участвовать в обсуждениях.

Для этого необходимо реализовать клиентскую и серверную части приложения, настроить хранение и обработку данных с помощью MongoDB, а также обеспечить удобный и понятный интерфейс для взаимодействия с системой.

### **3. Предлагаемое решение**

Проект реализуется в виде веб-приложения с использованием фреймворка Django на языке Python, что обеспечивает удобную и надёжную серверную часть. В качестве базы данных используется MongoDB, которая позволяет гибко хранить рецепты, комментарии, изображения и другую связанную информацию.

Приложение будет включать каталог хлеба с подробными рецептами, возможностью оставлять отзывы, комментарии и участвовать в обсуждениях. Решение ориентировано на удобство пользователей, лёгкость добавления нового контента и последующее масштабирование проекта.

### **4. Качественные требования к решению**

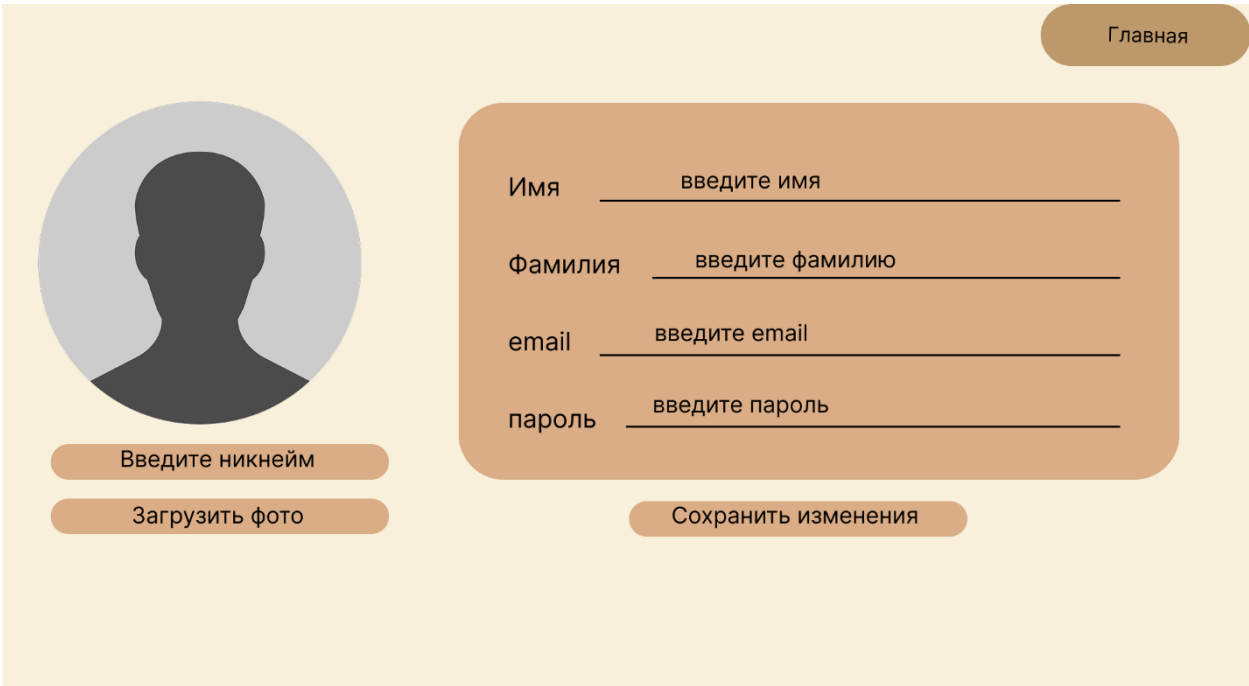
- Высокая производительность при работе с данными и визуализациях
- Удобный и понятный пользовательский интерфейс
- Надежность и устойчивость к ошибкам при взаимодействии с базой данных
- Актуальность и полнота отображаемой информации

## СЦЕНАРИЙ ИСПОЛЬЗОВАНИЯ

Перед началом разработки мы подробно проанализировали, как пользователи будут взаимодействовать с сайтом. В этом разделе представлены ключевые сценарии использования — типовые маршруты, по которым пользователи будут достигать своих целей.

Сценарии основаны на реальных потребностях целевой аудитории и учитывают навигационную структуру, элементы интерфейса и возможные действия. Такой подход позволяет нам спроектировать понятный и удобный интерфейс, в котором все функции работают логично и последовательно.

### 1. Макет UI



Макет UI формы регистрации пользователя. В верхнем правом углу находится кнопка «Главная». Слева расположена область для аватара: круглая рамка с серым силуэтом человека, под которой находятся две кнопки: «Введите никнейм» и «Загрузить фото». Справа — форма ввода данных, состоящая из четырех полей: «Имя», «Фамилия», «email» и «пароль», каждое с подсказкой «введите...». В нижнем правом углу формы находится кнопка «Сохранить изменения».

Рисунок 1 - Форма регистрации пользователя



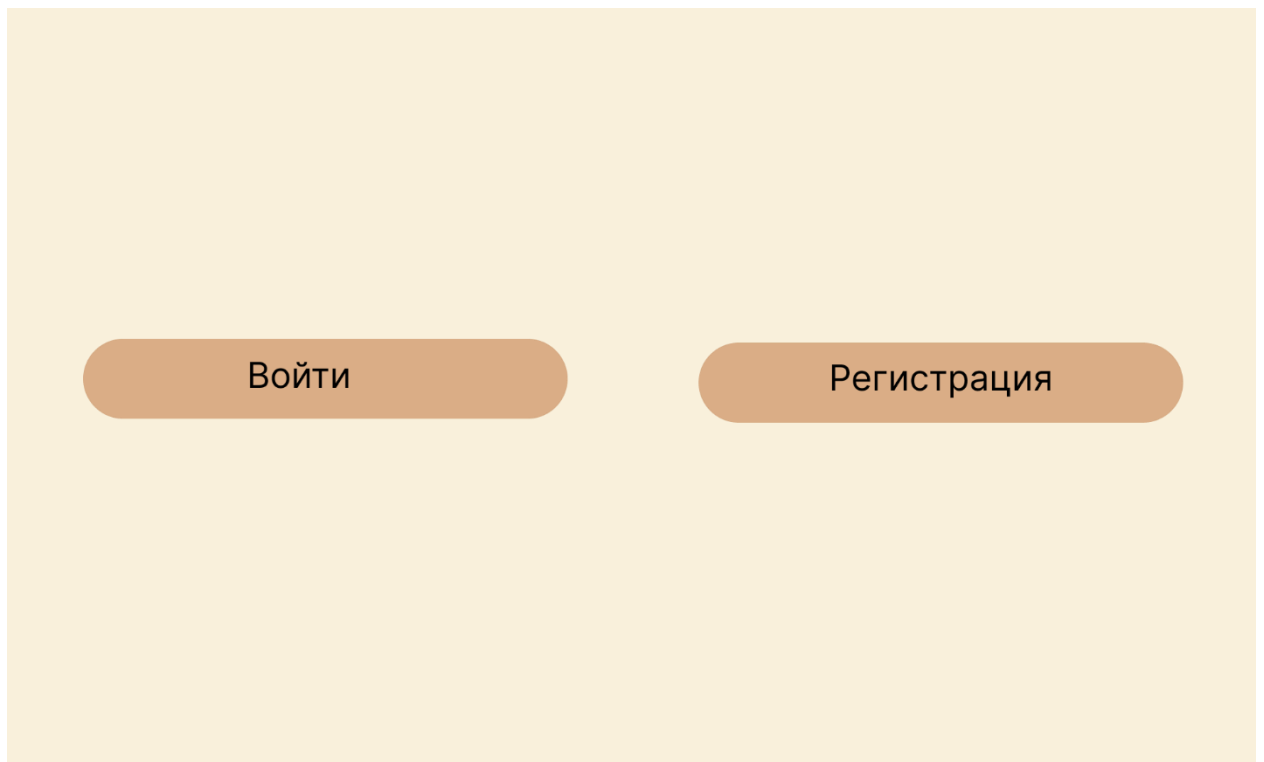


Рисунок 2 - Переход на страницу с регистрацией пользователя

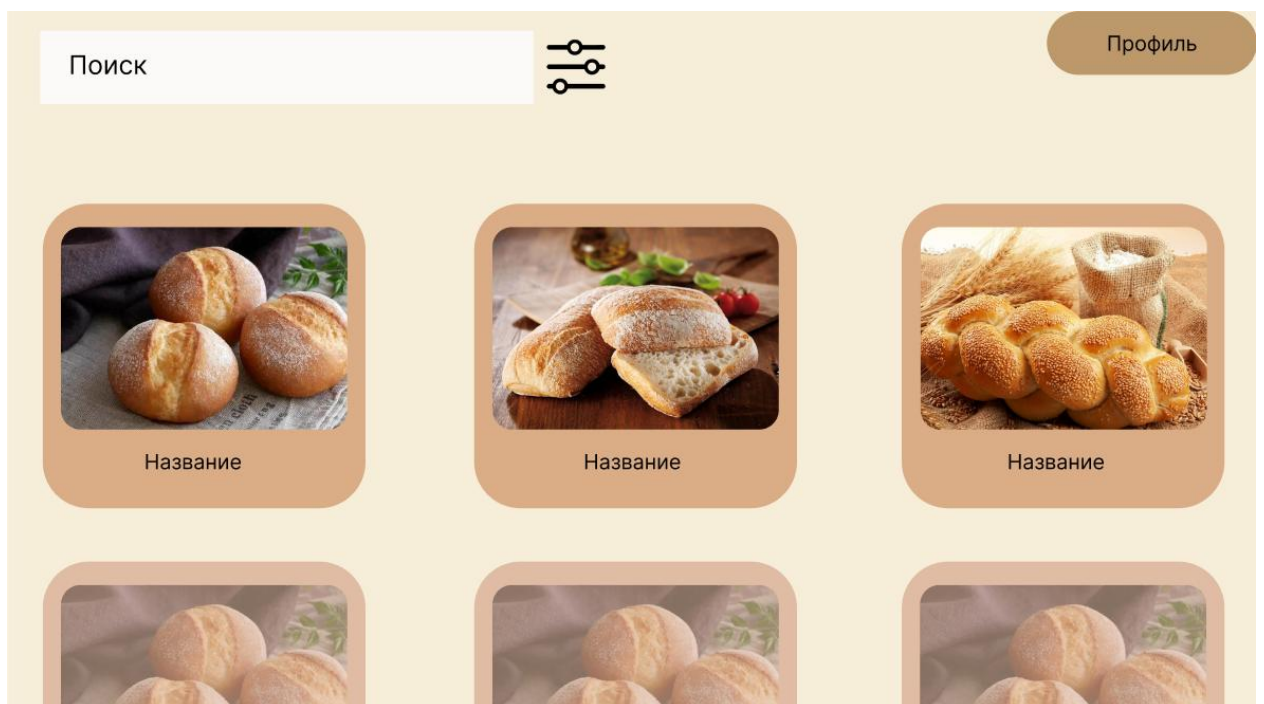


Рисунок 3 - Главная страница сайта

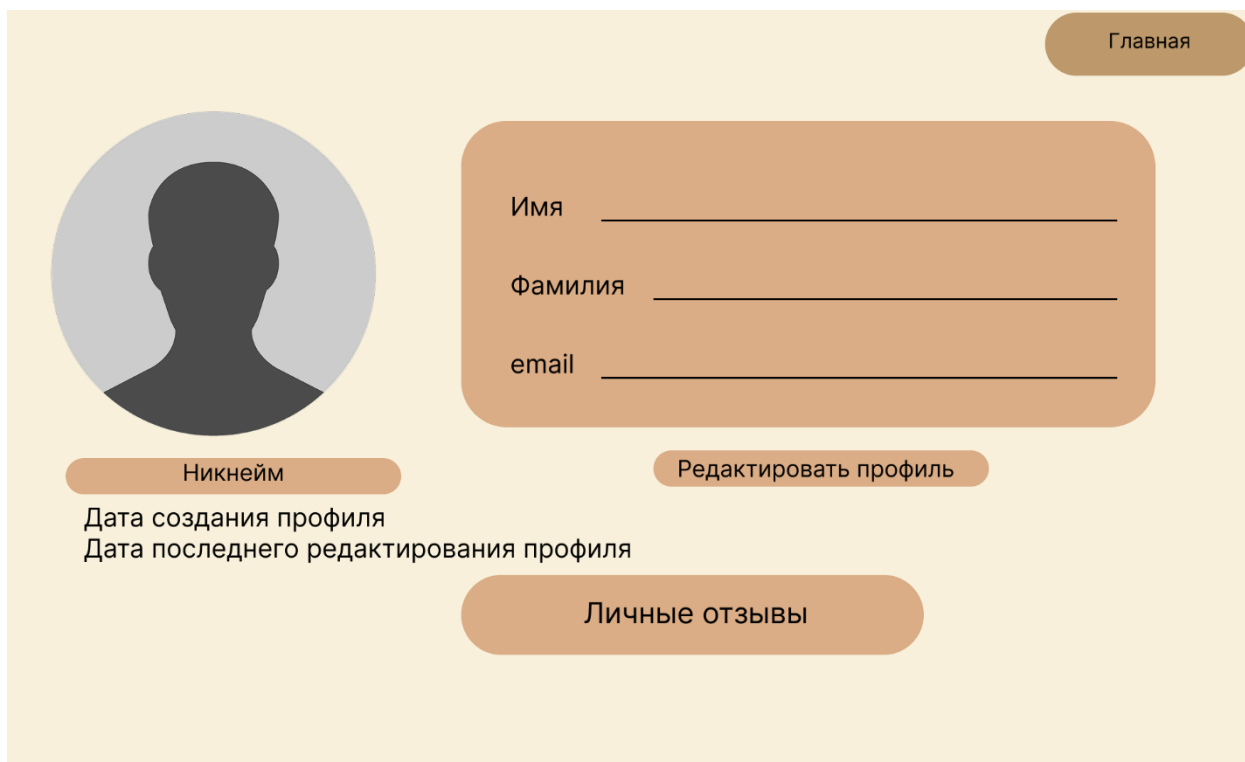


Рисунок 4 - Личный кабинет пользователя



Рисунок 5 - Редактирование данных пользователя

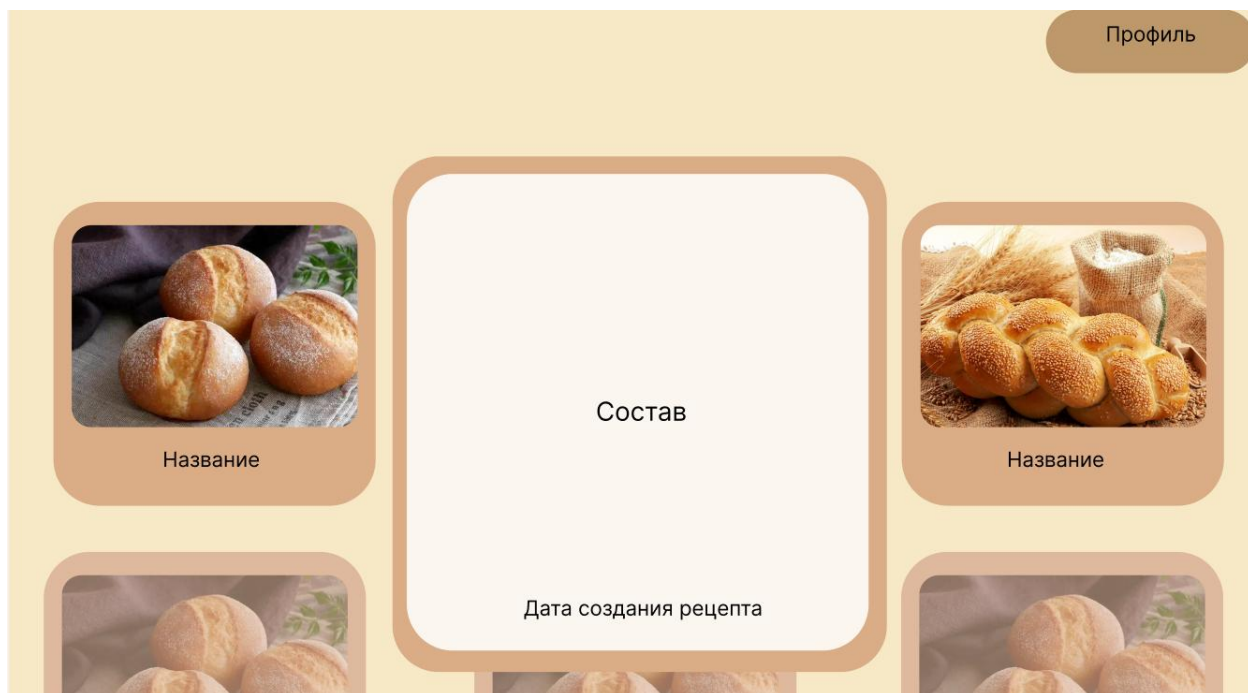


Рисунок 6 - Предварительное открытие карточки с составом

Главная



Имя \_\_\_\_\_

Фамилия \_\_\_\_\_

email \_\_\_\_\_

Никнейм

Редактировать профиль

Дата создания профиля

Дата последнего редактирования профиля

Личные отзывы



Иванов Иван, 23.02.2032, ★★★★★  
\*\*\*\*\*  
\*\*\*\*\*

Название

Перейти к отзыву



Текст отзыва

Название

Перейти к отзыву



Текст отзыва

Название

Перейти к отзыву

Рисунок 7 - Просмотр отзывов, которые оставил пользователь



Рисунок 8 – Просмотр всех отзывов на рецепт и добавление своего



Рисунок 9 - Оценка карточки пользователем



Рисунок 10 - Просмотр статистики отзывов

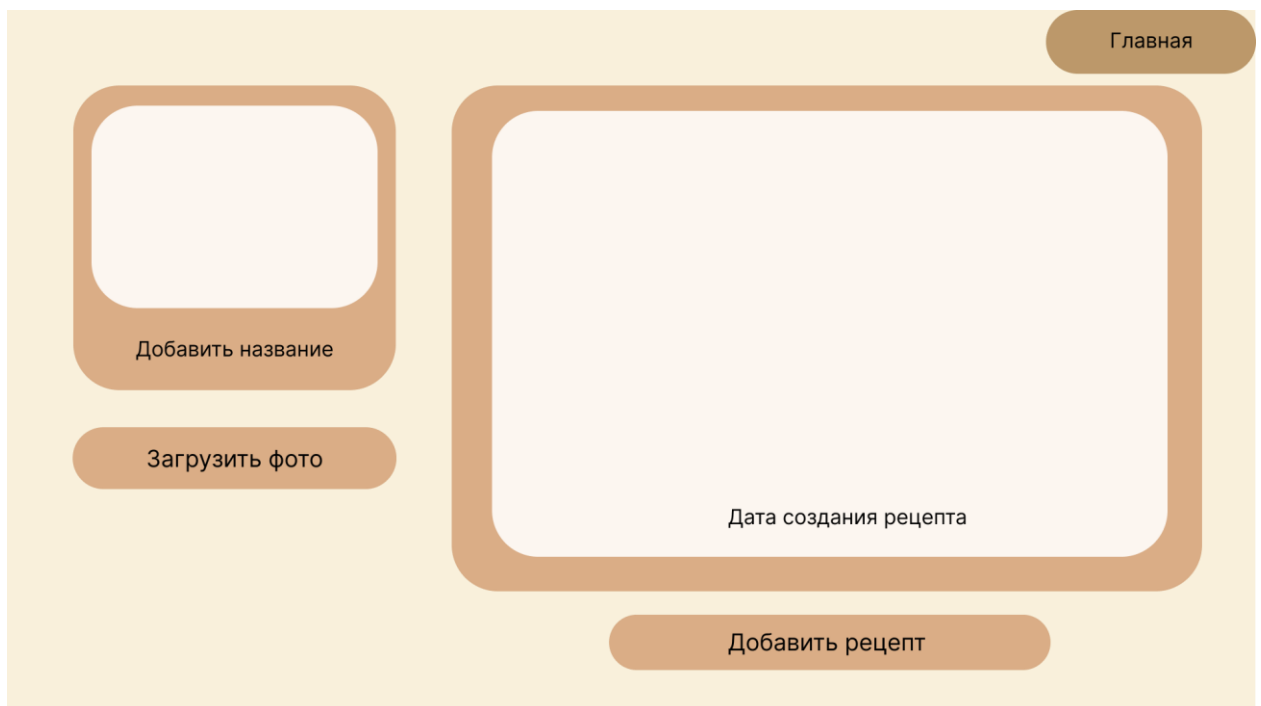


Рисунок 11 - Добавление нового рецепта администратором



Рисунок 12 - Просмотр всех отзывов



Рисунок 13 - Переход к конкретному отзыву

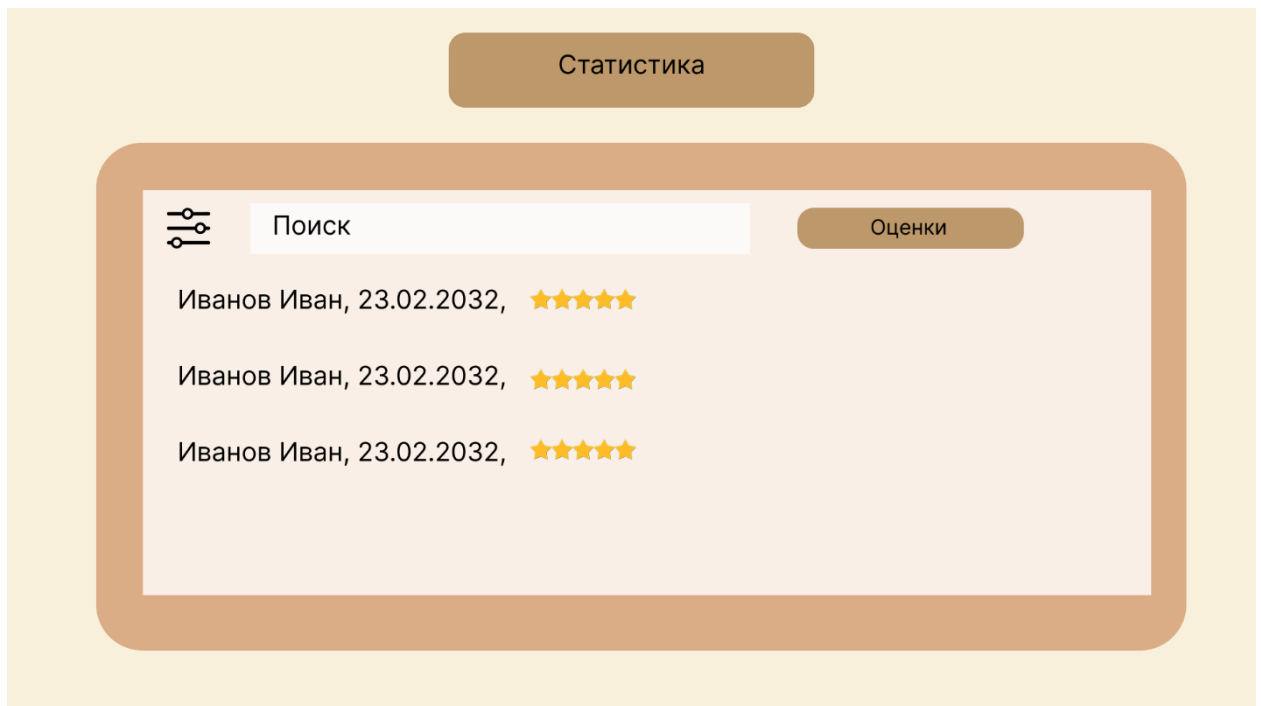


Рисунок 14 - Просмотр статистики оценок

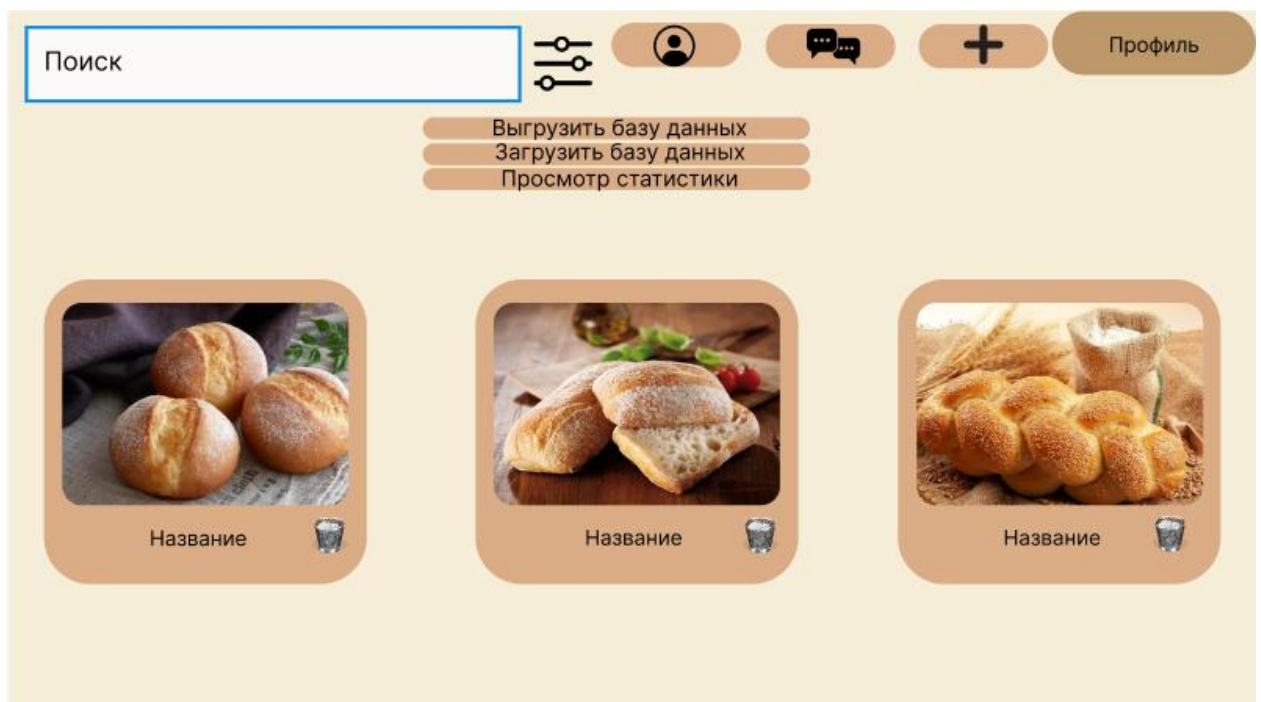


Рисунок 15 - Главная страница администратора





Рисунок 16 - Просмотр карточки с рецептом

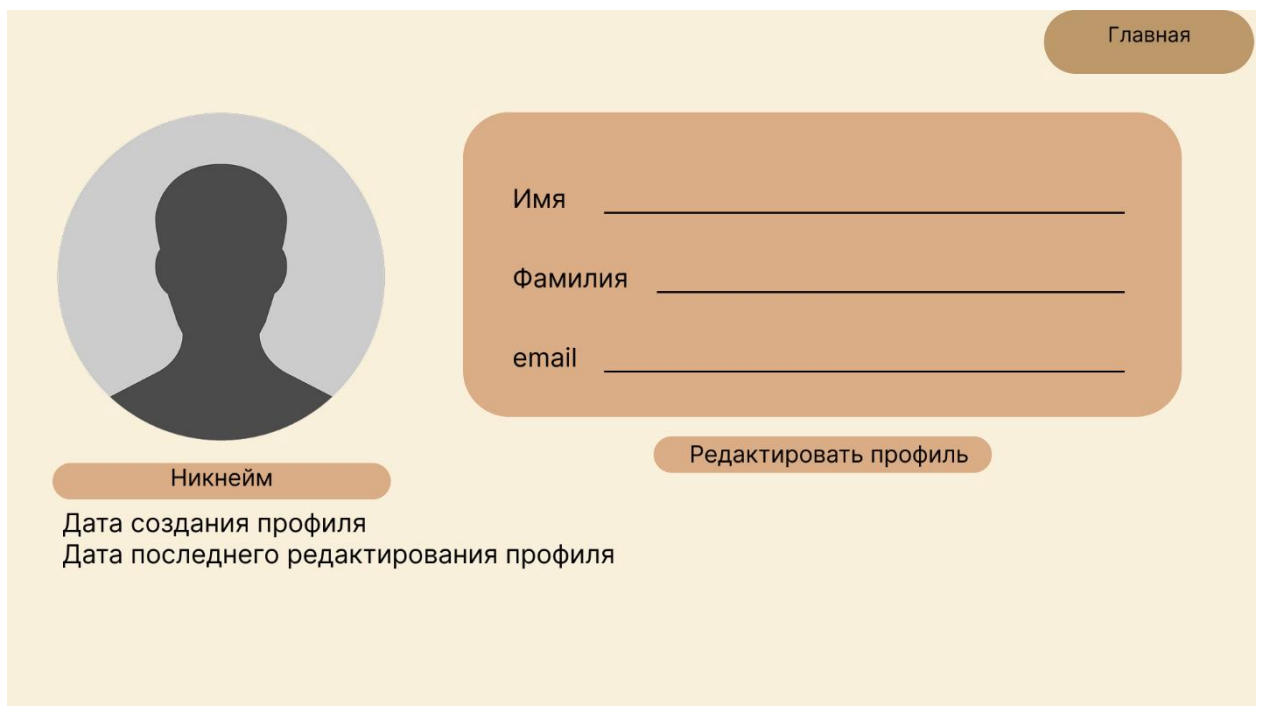


Рисунок 17 - Личный кабинет администратора

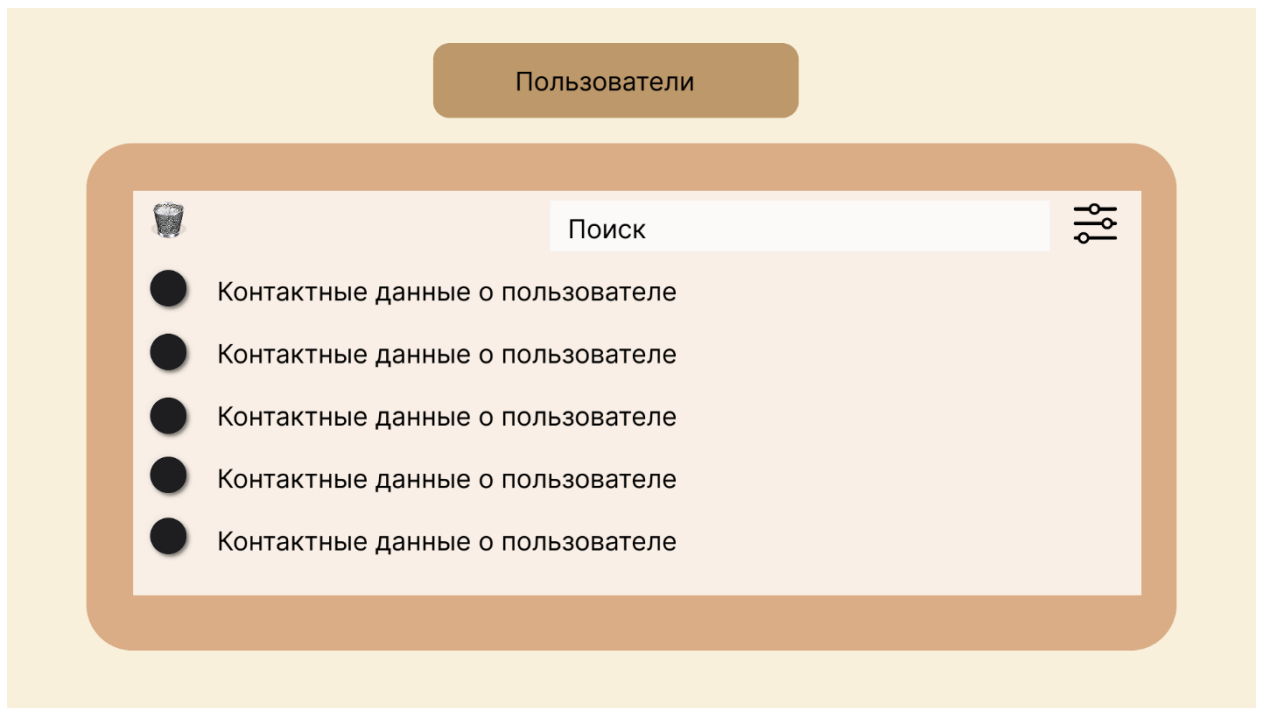


Рисунок 18 - Просмотр всех пользователей

Главная



Имя \_\_\_\_\_

Фамилия \_\_\_\_\_

email \_\_\_\_\_

Никнейм

Заблокировать пользователя

Личные отзывы



Иванов Иван, 23.02.2032, ★★★★★  
\*\*\*\*\*  
\*\*\*\*\*

Название

Перейти к отзыву

Рисунок 19 - Просмотр личного профиля пользователя

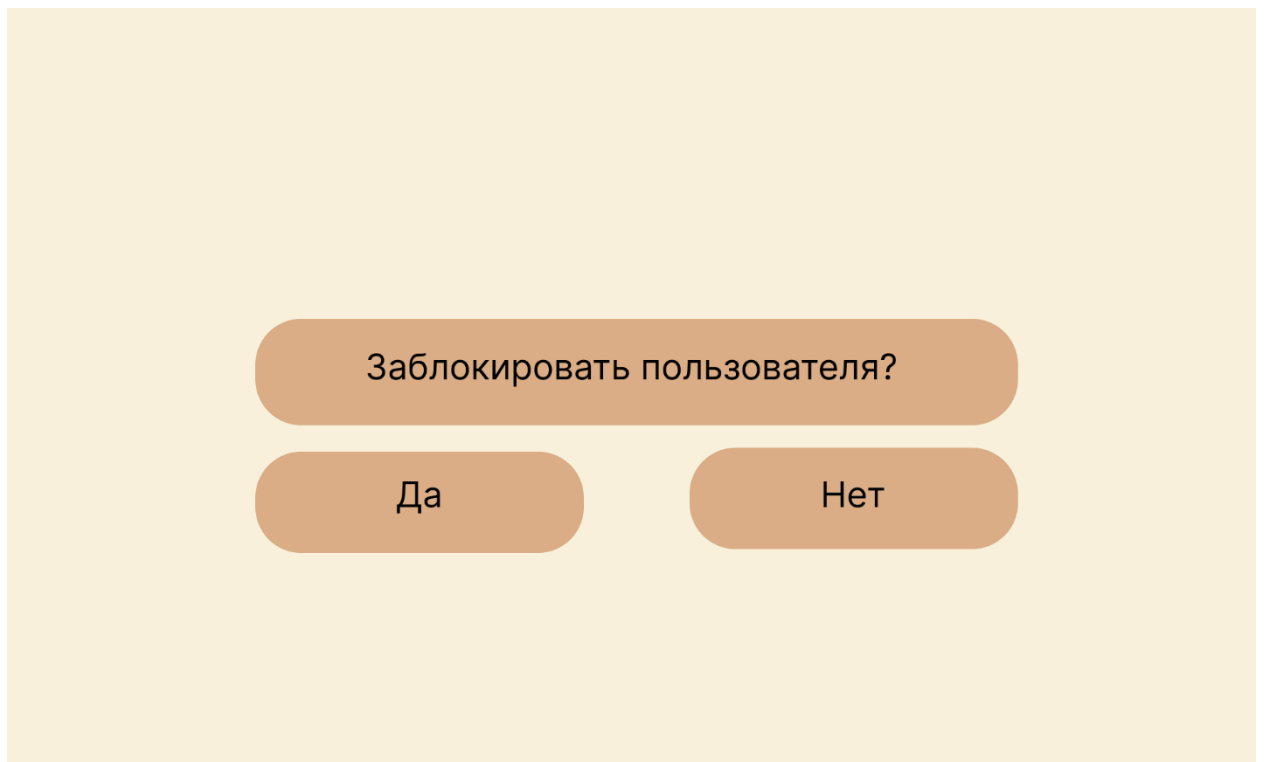


Рисунок 20 - Запрос на подтверждение операции блокировки пользователя



Рисунок 21 - Запрос на подтверждение операции удаления отзыва



Рисунок 22 - Запрос на подтверждение удаления пользователя



Рисунок 23 - Запрос на подтверждение удаления рецепта

## **2. Сценарий использования**

### **Сценарий использования №1 - Поиск рецепта на главной странице сайта.**

*Действующее лицо:* Пользователь.

*Основной сценарий:*

Сайт открывается.

Пользователь может нажать на кнопку "Поиск" и написать название хлеба или пролистать страницу до нужного ему рецепта.

Пользователь нажимает кнопку фильтрации и может отфильтровать рецепты по дате добавления, оценке

Пользователь наводит мышку на карточку и видит состав.

### **Сценарий использования №2 - Просмотр рецепта.**

*Действующее лицо:* Пользователь

*Основной сценарий:*

Пользователь может нажать на карточку и просмотреть рецепт, отзывы и оценку.

### **Сценарий использования №3 - Оценка и отзыв на рецепт**

*Действующее лицо:* Пользователь

*Основной сценарий:*

Пользователь может нажать на кнопку "Добавить отзыв" или "Добавить оценку"

Если пользователь зарегистрирован, то он оставляет отзыв или оценку, в ином случае, пользователь переходит на экран регистрации

Незарегистрированный пользователь вводит свои контактные данные

#### **Сценарий использования №4 - Взаимодействие с личным кабинетом, зарегистрированного пользователя**

*Действующее лицо:* Пользователь

*Основной сценарий:*

Пользователь может нажать на кнопку «Профиль»

Пользователь может нажать на кнопку «Редактировать профиль» и изменить свои контактные данные<sup>[LSEP]</sup>

Пользователь может просмотреть дату создания и последнего редактирования своего профиля

Пользователь может нажать на кнопку «Личные отзывы», в выпадающей вкладке он может просмотреть свои оставленные отзывы и перейти к выбранному

#### **Сценарий использования №5 - Добавление рецепта**

*Действующее лицо:* Администратор

*Основной сценарий:*

Администратор может нажать на кнопку "Добавить новый рецепт"

Администратор переходит в новое окно и добавляет информацию о рецепте<sup>[LSEP]</sup>

Администратор загружает рецепт на страницу

#### **Сценарий использования №6 - Удаление рецепта**

*Действующее лицо:* Администратор

*Основной сценарий:* <sup>[LSEP]</sup>Администратор может нажать на корзину и после подтверждения того, что он действительно намерен удалить рецепт, удаляет его

#### **Сценарий использования №7 - Массовый импорт/экспорт данных**

*Действующее лицо:* Администратор

*Основной сценарий:*

Администратор находится на главной странице, нажимает кнопку «Выгрузить базу данных», добавляет файл с данными и нажимает кнопку «Загрузить базу данных»

### **Сценарий использования №8 - Просмотр данных о пользователях и об отзывах**

*Действующее лицо:* Администратор

*Основной сценарий:*

Администратор находится на главной странице

Администратор нажимает кнопку со значком пользователей и переходит на страницу с таблицей, в которой видит всех пользователей сайта

Администратор может перейти к пользователю в профиль и может просмотреть данные<sup>[1]</sup><sub>SEP</sub>

Администратор нажимает кнопку со значком сообщений, и может просмотреть все отзывы на сайте, выбирает отзыв, переходит на страницу с выбранным отзывом и, при необходимости, после подтверждения удаления, удаляет его.

### **Сценарий использования №9 - Просмотр статистики о пользователях**

*Действующее лицо:* Администратор

*Основной сценарий:*

Администратор находится на главной странице

Администратор нажимает кнопку «Просмотр статистики» и получает возможность выбрать по какому критерию он будет просматривать статистику, после выбора критерия перед ним появляется таблица с выбранными данными просмотреть статистику пользователей.



### **3. Преобладающие операции**

В предлагаемом решении преобладающими будут операции чтения.

Из-за специфики сайта большинство пользователей в первую очередь будут просматривать рецепты, комментарии к ним, при этом зачастую пользователь не оставляет отзывы и или комментарии, что значит, что в основном будут происходить только операции чтения. Также новые рецепты могут добавлять только администраторы, а не все пользователи, что также ограничивает количество записей.

# Модель данных

## 1. Нереляционная модель данных

### 1.1. Графическое представление

| User            |                  |
|-----------------|------------------|
| PK              | <u>UserID</u>    |
| String          | Name             |
| String          | Surname          |
| String          | Nickname         |
| Timestamp       | Create_Date      |
| Timestamp       | Last_Redact_Date |
| String          | email            |
| Array [Integer] | CommentIDs       |
| String          | hash_Password    |

| Recipe          |                 |
|-----------------|-----------------|
| PK              | <u>RecipeID</u> |
| String          | Title           |
| String          | Composition     |
| Timestamp       | Date            |
| String          | Recipe          |
| Integer         | Rate            |
| Array [Integer] | CommentIDs      |

| Comment   |                  |
|-----------|------------------|
| PK        | <u>CommentID</u> |
| String    | Username         |
| String    | Text             |
| Integer   | Rating           |
| Timestamp | Date             |
| Integer   | RecipeID         |
| String    | Title            |

Рисунок 24 - Представления нереляционной модели

### 1.2. Описание назначений коллекций, типов данных и сущностей

Коллекция User содержит в себе имя, фамилию, никнейм пользователя, которые являются строками и отображаются в профиле пользователя. Также хранится дата и время создания и последнего изменения. Для каждого пользователя хранится массив с id всех его комментариев и пароль.

Коллекция Recipe хранит в себе информацию о рецептах хлеба: название, описание, рецепт – являются строковыми типами, и описывают основную информацию о карточке. Для рецепта определен рейтинг, дата создания, а также массив с id комментариями к рецепту.

Коллекция Comment отвечает за комментарии пользователей, в ней хранится вся информация о них, такая как: имя пользователя, оставившего комментарий, текст комментария, название(строковые

поля), а также дата создания, оценки комментария другими пользователями и его id.

### 1.3. Оценка удельного объема информации, хранимой в модели

Рассмотрим теперь рост объема базы данных от одной переменной. В среднем на нашем сайте будет 100 рецептов, n пользователей и 2n комментариев. Тогда получаем с учетом объемов каждой сущности следующую зависимость:  $g(n) = 100 \cdot 960 + 237 \cdot n + 567 \cdot 2 \cdot n = 96000 + 1371 \cdot n$  байт.

### 1.4. Запросы к модели, с помощью которых реализуется сценарии использования

|                                                                                                  |                                                                                                                                                                                                                                                                      |
|--------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Регистрация<br>пользователя                                                                      | <pre>db.User.insertOne({   UniqueID: ObjectId(),   Name: "Иван",   Surname: "Иванов",   Nickname: "IvanCool",   Create_Date: new Date(),   Last_Redact_Date: new Date(),   email: "ivan@example.com",   CommentIDs: [],   hash_Password: "hashed_password" });</pre> |
| Авторизация<br>пользователя (поиск по<br>email и паролю)<br><br>Просмотр профиля<br>пользователя | <pre>db.User.findOne({   email: "ivan@example.com",   hash_Password: "hashed_password" });</pre>                                                                                                                                                                     |
| Просмотр профиля<br>пользователя                                                                 | <pre>db.User.findOne(   { UniqueID:     ObjectId("id_пользователя") },   { hash_Password: 0 } // Исключаем   поле с паролем );</pre>                                                                                                                                 |
| Редактирование<br>профиля пользователя                                                           | <pre>db.User.updateOne(   { UniqueID:     ObjectId("id_пользователя") },   {</pre>                                                                                                                                                                                   |

|                                           |                                                                                                                                                                                                                                                                                                                            |
|-------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                                           | <pre>         \$set: {             Name: "Иван",             Surname: "Петров",             Nickname: "IvanUpdated",             email: "ivan.new@example.com",             Last_Redact_Date: new Date()         }     } }; </pre>                                                                                         |
| Добавление нового рецепта                 | <pre> db.Recipe.insertOne({     RecipeID: ObjectId(),     Title: "Борщ",     Composition: "Свекла, капуста, картошка, мясо",     Date: new Date(),     Recipe: "Шаг 1: Нарезьте овощи. Шаг 2: Варите 40 минут...",     Rate: 0,     CommentIDs: [] }); </pre>                                                              |
| Редактирование рецепта                    | <pre> db.Recipe.updateOne(     { RecipeID: ObjectId("id_рецепта") },     {         \$set: {             Title: "Борщ по- украински",             Composition: "Свекла, капуста, картошка, мясо, чеснок",             Recipe: "Обновленный способ приготовления...",             Date: new Date()         }     } ); </pre> |
| Удаление рецепта                          | <pre> db.Recipe.deleteOne({ RecipeID: ObjectId("id_рецепта") }); </pre>                                                                                                                                                                                                                                                    |
| Просмотр списка рецептов                  | <pre> db.Recipe.find({}, { Recipe: 0 }).pretty(); // Исключаем поле Recipe, чтобы не загружать всю информацию в списке </pre>                                                                                                                                                                                              |
| Просмотр одного рецепта (с комментариями) | <pre> db.Recipe.aggregate([     { \$match: { RecipeID: ObjectId("id_рецепта") } },     {         \$lookup: {             from: "Comment",             localField: "RecipeID",             foreignField: "RecipeID",             as: "Comments"         }     } ]); </pre>                                                  |

|                                     |                                                                                                                                                                                                                                                                                                                                                               |
|-------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                                     | <pre>         }     }).pretty(); </pre>                                                                                                                                                                                                                                                                                                                       |
| Добавление<br>комментария к рецепту | <pre> let commentID = ObjectId(); db.Comment.insertOne({   UniqueID: commentID,   Username: "IvanCool",   Text: "Очень вкусный борщ!",   Rating: 5,   Date: new Date(),   RecipeID: ObjectId("id_рецепта"),   Title: "Лучший борщ" }); db.Recipe.updateOne(   { RecipeID:     ObjectId("id_рецепта") },   { \$push: { CommentIDs:     commentID } } ); </pre> |
| Редактирование<br>комментария       | <pre> db.Comment.updateOne(   { UniqueID:     ObjectId("id_комментария") },   { \$set: { Text: "Обновленный комментарий", Rating: 4, Date: new Date() } } ); </pre>                                                                                                                                                                                           |
| Удаление комментария                | <pre> db.Comment.deleteOne({ UniqueID:   ObjectId("id_комментария") }); db.Recipe.updateOne(   { RecipeID:     ObjectId("id_рецепта") },   { \$pull: { CommentIDs:     ObjectId("id_комментария") } } ); </pre>                                                                                                                                               |
| Поиск рецептов по<br>названию       | <pre> db.Recipe.find({ Title: { \$regex:   "борщ", \$options: "i" } }).pretty(); </pre>                                                                                                                                                                                                                                                                       |
| Фильтрация рецептов<br>по рейтингу  | <pre> db.Recipe.find({ Rate: { \$gte:   4 } }).pretty(); </pre>                                                                                                                                                                                                                                                                                               |
| Выставление рейтинга<br>рецепту     | <pre> db.Recipe.updateOne(   { RecipeID:     ObjectId("id_рецепта") },   { \$set: { Rate: 5 } } ); </pre>                                                                                                                                                                                                                                                     |

Таблица 1 - Пример запросов к нереляционной бд

## 2. Реляционная модель данных

### 2.1. Графическое представление

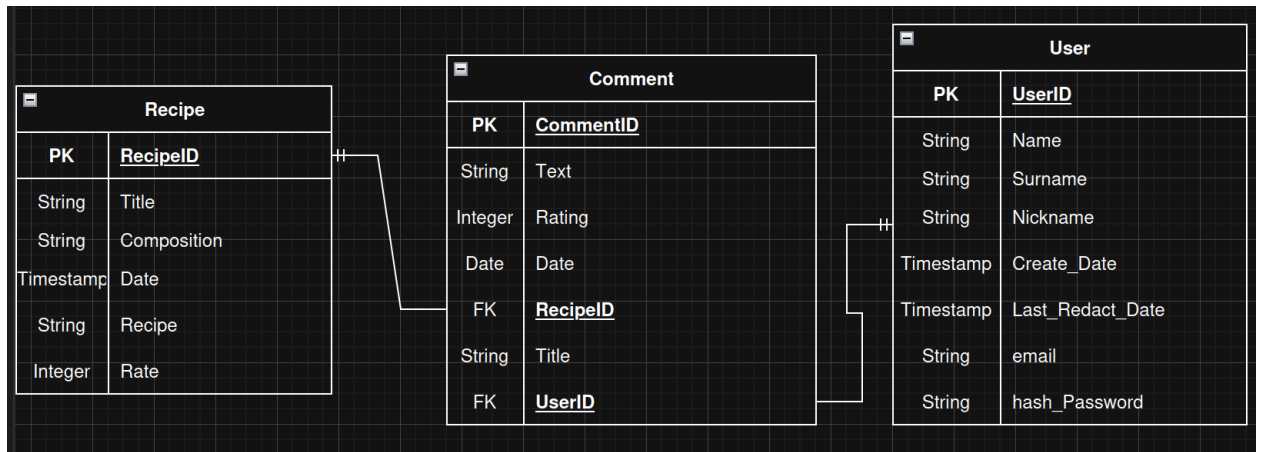


Рисунок 25 - Представление реляционной модели

### 2.2. Описание назначений коллекций, типов данных и сущностей

Таблица User хранит информацию о пользователях системы. В ней содержатся строковые поля для имени, фамилии и никнейма пользователя, которые отображаются в профиле. Также фиксируется дата создания аккаунта и дата последнего редактирования. Для каждого пользователя сохраняется адрес электронной почты и хэшированный пароль. Идентификатор пользователя (UserID) используется как первичный ключ и связывается с комментариями, оставленными пользователем.

Таблица Recipe содержит данные о рецептах хлеба. Для каждого рецепта указывается название, состав, текст рецепта (все строкового типа), а также дата добавления и числовой рейтинг. Рецепт идентифицируется по RecipeID (первичный ключ). Комментарии к рецептам реализованы через внешнюю связь с таблицей Comment.

Таблица Comment отвечает за хранение комментариев пользователей. В ней содержатся текст комментария, его название, дата создания,

оценка (рейтинг) и внешний ключи, указывающие на рецепт (RecipeID) и пользователя (UserID), к которым относится комментарий. Уникальный идентификатор комментария (CommentID) используется как первичный ключ.

### 2.3. Оценка удельного объема информации, хранимой в модели

Рассмотрим теперь рост объема базы данных от одной переменной (количество пользователей). В среднем на нашем сайте будет 100 рецептов,  $n$  пользователей и  $2n$  комментариев. Тогда получаем с учетом объемов каждой сущности следующую зависимость:  $g(n) = 100 \cdot 840 + 117 \cdot n + 564 \cdot 2 \cdot n = 84000 + 1245 \cdot n$  байт.

### 2.4. Запросы к модели, с помощью которых реализуется сценарии использования

|                                                          |                                                                                                                                                                                                                         |
|----------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Регистрация<br>пользователя                              | INSERT INTO User (Name, Surname, NickName, email, hash_Password)<br>VALUES ('Имя', 'Фамилия', 'Никнейм',<br>'email@example.com', 123456);                                                                               |
| Авторизация<br>пользователя (поиск по<br>email и паролю) | SELECT * FROM User<br>WHERE email = 'email@example.com' AND<br>hash_Password = 123456;                                                                                                                                  |
| Просмотр профиля<br>пользователя                         | SELECT Name, Surname, NickName,<br>Create_Date, email<br>FROM User<br>WHERE id = 1;                                                                                                                                     |
| Редактирование<br>профиля пользователя                   | UPDATE User<br>SET<br>Name = 'Новое имя',<br>Surname = 'Новая фамилия',<br>NickName = 'Новый ник',<br>email = 'new@example.com',<br>hash_Password = 654321,<br>Last_Redact_Date =<br>CURRENT_TIMESTAMP<br>WHERE id = 1; |
| Добавление нового<br>рецепта                             | INSERT INTO Recipe (Title,<br>Composition, Recipe, Rate)<br>VALUES ('Название рецепта',<br>'Ингредиенты', 'Инструкция', 5);                                                                                             |

|                                           |                                                                                                                                                                                                                           |
|-------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                                           |                                                                                                                                                                                                                           |
| Редактирование рецепта                    | <pre> UPDATE Recipe SET     Title = 'Новое название',     Composition = 'Новые ингредиенты',     Recipe = 'Новая инструкция',     Rate = 4,     Date = CURRENT_TIMESTAMP WHERE id = 1; </pre>                             |
| Удаление рецепта                          | <pre> DELETE FROM Recipe WHERE id = 1; </pre>                                                                                                                                                                             |
| Просмотр списка рецептов                  | <pre> SELECT id, Title, Date, Rate FROM Recipe ORDER BY Date DESC; </pre>                                                                                                                                                 |
| Просмотр одного рецепта (с комментариями) | <pre> SELECT * FROM Recipe WHERE id = 1; SELECT     Comment.Text,     Comment.Rating,     Comment.Date,     User.NickName AS Author FROM Comment JOIN User ON Comment.UserID = User.id WHERE Comment.RecipeID = 1; </pre> |
| Добавление комментария к рецепту          | <pre> INSERT INTO Comment (Text, Rating, RecipeID, Title, UserID) VALUES ('Текст комментария', 5, 1, 'Заголовок', 1); </pre>                                                                                              |
| Редактирование комментария                | <pre> UPDATE Comment SET     Text = 'Новый текст',     Rating = 4,     Date = CURRENT_TIMESTAMP WHERE id = 1 AND UserID = 1; </pre>                                                                                       |
| Удаление комментария                      | <pre> DELETE FROM Comment WHERE id = 1 AND UserID = 1; </pre>                                                                                                                                                             |
| Поиск рецептов по названию                | <pre> SELECT * FROM Recipe WHERE Title LIKE '%Название%'; </pre>                                                                                                                                                          |
| Фильтрация рецептов по рейтингу           | <pre> SELECT * FROM Recipe WHERE Rate &gt;= 4; </pre>                                                                                                                                                                     |



|                                 |                                                       |
|---------------------------------|-------------------------------------------------------|
| Выставление рейтинга<br>рецепту | <pre> UPDATE Recipe SET Rate = 5 WHERE id = 1; </pre> |
|---------------------------------|-------------------------------------------------------|

Таблица 2 - Пример запросов к реляционной бд

### 3. Сравнение моделей

#### 3.1. Удельный объем информации

| Параметр         | NoSQL (MongoDB)        | SQL (Реляционная)      |
|------------------|------------------------|------------------------|
| Формула объема   | $96000 + 1371 \cdot n$ | $84000 + 1245 \cdot n$ |
| Объем при C=50   | 161 КБ                 | 143 КБ                 |
| Избыточность     | 17%                    | 4%                     |
| Зависимость от C | Линейная (1371)        | Линейная (1245)        |

Таблица 3 - Сравнение объема в моделях

#### 3.2. Запросы по отдельным юзкейсам

*Сценарий использования №1 - Поиск рецепта на главной странице сайта.*

- SQL: 1 запрос
- MongoDB: 1 запрос

*Сценарий использования №2 - Просмотр рецепта*

- SQL: 2 запроса
- MongoDB: 2 запроса

*Сценарий использования №3 - Оценка и отзыв на рецепт*

- SQL: 3 запроса
- MongoDB: 3 запроса

*Сценарий использования №4 - Взаимодействие с личным кабинетом, зарегистрированного пользователя*

- SQL: 3 запроса
- MongoDB: 3 запроса

*Сценарий использования №5 - Добавление рецепта*

- SQL: 1 запрос
- MongoDB: 1 запрос

*Сценарий использования №6 - Удаление рецепта*

- SQL: 1 запрос
- MongoDB: 1-2 запроса

*Сценарий использования №7 - Массовый импорт/экспорт данных*

- SQL: 1 запрос
- MongoDB: 1 запрос

*Сценарий использования №8 - Просмотр данных о пользователях и об отзывах*

- SQL: 3 запроса
- MongoDB: 3 запроса

*Сценарий использования №9 - Просмотр статистики о пользователях*

- SQL: 1 запрос
- MongoDB: 1 запрос

#### **4. Вывод**

SQL база данных лучше подходит для наших задач. Она занимает имеет меньший вес и меньшую избыточность. Количество запросов должно быть относительно одинаковым.

## **3. РАЗРАБОТАННОЕ ПРИЛОЖЕНИЕ**

### **3.1. Краткое описание**

Приложение представляет собой современное веб-решение, состоящее из 3 основных компонентов: Frontend(клиентская часть) реализованный на html + css, Backend(серверная часть, отвечающая за запросы к бд и передаче этих данных frontend) реализованный с использованием фреймворка Django, а также сама база данных MongoDB, реализующая коллекции для хранения и записи данных.

### **3.2. Используемые технологии**

Frontend:

- JavaScript
- HTML5
- CSS3

Backend:

- Python
- Django
- Django

Инфраструктура:

- Docker
- Docker Compose
- MongoDB
- Git

### 3.3. Снимки экрана приложения

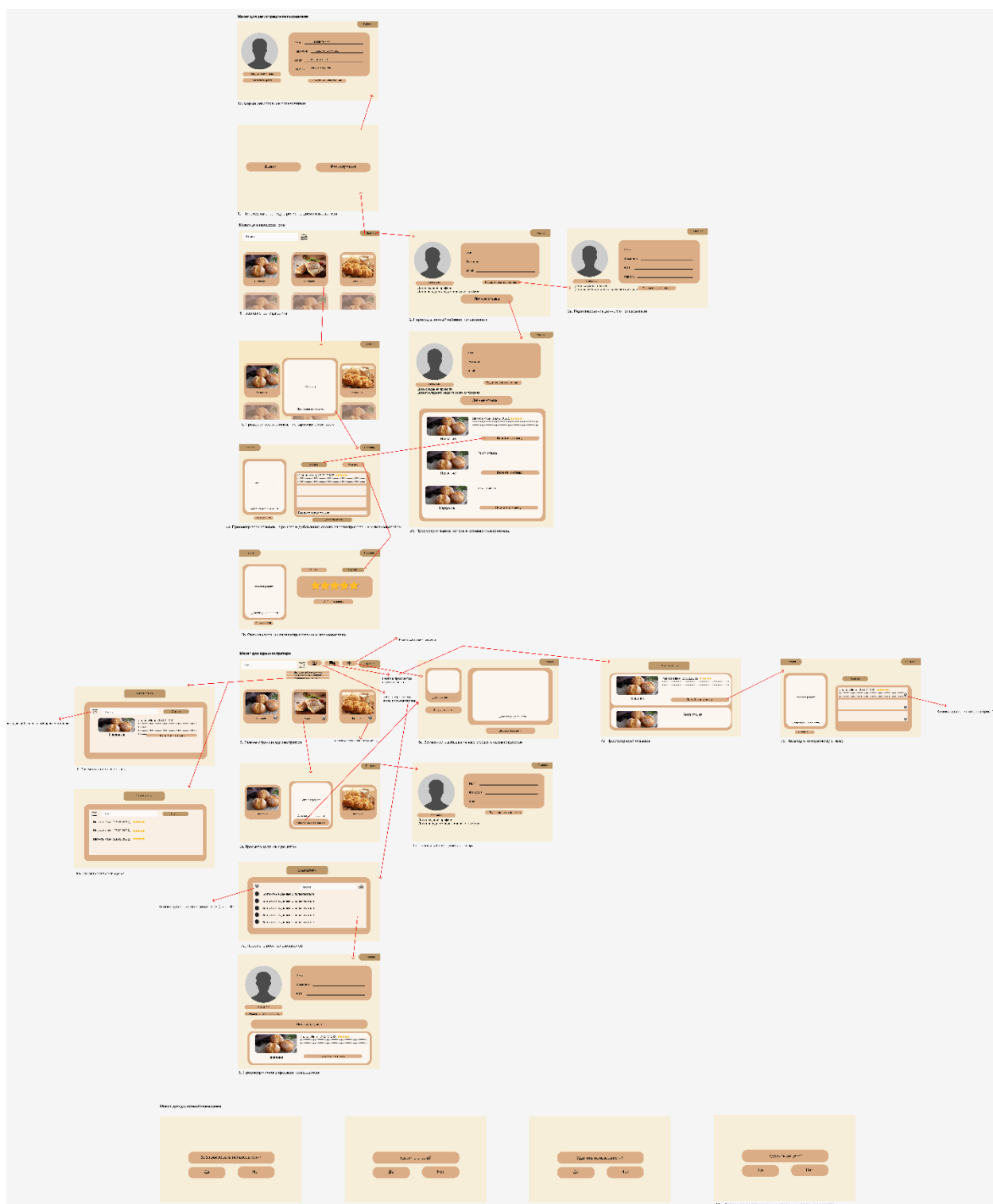


Рисунок 26 – Экрана переходов по сайту

## **ВЫВОДЫ**

### **4.1. Достигнутые результаты**

- Создано приложение с ограниченным функционалом и интуитивно понятным интерфейсом
- Обеспечена высокая производительность при работе с большими объемами данных
- Реализована удобная система поиска и фильтрации информации

### **4.2. Недостатки и пути для улучшения полученного решения**

- Добавление возможности загружать свои рецепты хлеба
- Добавлении системы комментирования и простановки рейтинга
- Добавление возможности модерации

### **4.3. Будущее развитие решения**

- Добавление системы комментариев и обсуждений
- Создание мобильной версии приложения
- Добавление многоязычности
- Интеграция с внешними API для получения дополнительной информации

## ЗАКЛЮЧЕНИЕ

В ходе разработки веб-приложения был успешно создан современный сайт-каталог, посвящённый рецептам хлеба. Приложение демонстрирует эффективное использование документо-ориентированной базы данных MongoDB для хранения и управления информацией о рецептах, ингредиентах и пользователях.

Ключевыми достижениями проекта являются:

1. Разработка удобного пользовательского интерфейса для просмотра и поиска рецептов хлеба
2. Реализация надёжной системы хранения данных с использованием MongoDB
3. Внедрение механизма авторизации пользователей, обеспечивающего безопасность и персонализацию
4. Построение архитектуры, позволяющей масштабировать и дополнять функциональность в будущем

Использование MongoDB позволило гибко и эффективно структурировать данные, что особенно важно для динамически развивающегося контента, связанного с кулинарными рецептами. Архитектура проекта предусматривает возможность интеграции новых функций без значительной переработки системы.

Несмотря на уже реализованный функционал, проект имеет значительный потенциал для дальнейшего развития. В числе запланированных улучшений — добавление системы оценок и комментариев к рецептам, что повысит вовлечённость пользователей и качество контента.

В целом, проект успешно демонстрирует возможности современных веб-технологий и подходов к разработке пользовательских систем с

ориентиром на удобство, масштабируемость и дальнейшее развитие функционала.



## ПРИЛОЖЕНИЯ

### 1. Документация по сборке и развертыванию приложения

#### 1. Системные требования

- Операционная система: Ubuntu 22.04 или новее
- Docker и Docker Compose

#### 2. Подготовка к установке

Клонирование репозитория:

```
git clone https://github.com/moevm/nosql1h25-bread.git  
cd nosql1h25-bread
```

Настройка переменных окружения: отредактируйте файл `.env` в соответствии с вашими требованиями

#### 3. Сборка и запуск

Сборка контейнеров:

```
docker-compose build --no-cache
```

Запуск приложения:

```
docker-compose up
```

#### 4. Доступ к сервисам

Frontend: <http://localhost:8000>

## ЛИТЕРАТУРА

Официальная документация:

- 1) Документация к фреймворку Django – <https://www.djangoproject.com/>
- 2) Документация к базе данных MongoDB – <https://www.mongodb.com/>
- 3) Документация к DjongoORM – <https://www.djongomapper.com/>

Ссылка на репозиторий с проектом -

<https://github.com/moevm/nosql1h25-bread>